



CPE 4222757 ADVANCED TOPICS IN DATABASE SYSTEMS

**DEVELOPMENT OF AN AI-POWERED MOVIE
RECOMMENDATION SYSTEM INFRASTRUCTURE
USING POSTGRESQL PGVECTOR**

Nisa Begüm Burucu

Student ID: 64210025

TABLE OF CONTENTS

1. Introduction
2. Project Objectives
3. Technologies Used
4. Dataset Description
5. Database Design
6. Data Import and Validation in PostgreSQL
 - 6.1 Verification of Total Number of Movies
 - 6.2 Verification of Total Number of Ratings
 - 6.3 Examination of Sample Movie Records
 - 6.4 Analysis of the Most Frequently Rated Movies
 - 6.5 Analysis of the Highest-Rated Movies
 - 6.6 Genre Distribution Analysis
 - 6.7 Verification of the Embedding Column
7. pgvector Extension
8. The Embedding Concept
9. Creation of the Embedding Column
10. Embedding Generation Using Python
11. Python Code
12. Explanation of the Python Code
 - 12.1 Importing the Pandas Library
 - 12.2 Importing the Sentence Transformer Library
 - 12.3 Loading the Model
 - 12.4 Reading the CSV File
 - 12.5 Displaying the Number of Movies
 - 12.6 Selecting Sample Movies
 - 12.7 Generating Embeddings
 - 12.8 Displaying the Results
13. Additional Explanation of the Python Workflow
14. Challenges Encountered
15. Skills and Knowledge Gained
16. Real-World Applications
17. Conclusion
18. References

1. Introduction

In this project, an infrastructure for AI-powered movie recommendation systems was developed using pgvector, an advanced extension for PostgreSQL. Today, platforms such as Netflix, YouTube, Spotify, and Amazon rely on similarity calculations and artificial intelligence techniques to provide personalized recommendations to their users. At the core of these systems are numerical vector representations known as embeddings.

The primary objective of this project is to demonstrate that PostgreSQL is not only a traditional relational database management system but can also be integrated with modern artificial intelligence applications.

To achieve this goal, the MovieLens dataset was utilized, the pgvector extension was installed on PostgreSQL, movie data was imported into the database, and embeddings were generated from movie titles using Python.

2. Project Objectives

The objectives of this project are as follows:

- To utilize an advanced PostgreSQL extension
- To explore pgvector technology
- To store movie-related data in a PostgreSQL environment
- To generate embeddings using artificial intelligence models
- To establish the foundation of a recommendation system
- To demonstrate the applicability of PostgreSQL in AI-driven applications

3. Technologies Used

The following technologies were used throughout the project.

PostgreSQL

PostgreSQL served as the relational database management system used to store and manage data.

Neon

Neon is a cloud-based PostgreSQL service. The database environment for this project was hosted on Neon.

DBeaver

DBeaver was used for database management and execution of SQL queries.

Python

Python was used for embedding generation and data processing tasks.

Pandas

The Pandas library was used for reading CSV files and handling data manipulation operations.

Sentence Transformers

Sentence Transformers was utilized to convert movie titles into vector representations known as embeddings.

pgvector

The pgvector extension was used to store vector data types and perform similarity calculations directly within PostgreSQL.

4. Dataset Description

The MovieLens Small Dataset was selected for this project.

The dataset consists of the following files:

- movies.csv
- ratings.csv
- tags.csv
- links.csv

Among these files, the following two datasets were primarily used.

movies.csv

This file contains movie titles and genre information.

Total number of records:

9742 movies

ratings.csv

This file contains user ratings assigned to movies.

Total number of records:

100,836 ratings

5. Database Design

Two primary tables were created during the implementation of this project.

Movies Table

movies

- movieid
- title
- genres
- embedding

This table stores movie-related information.

The embedding column was created using the pgvector data type.

Ratings Table

ratings

- userid
- movieid
- rating
- timestamp

This table stores ratings provided by users for movies.

6. Data Import and Validation in PostgreSQL

The movies.csv and ratings.csv files from the MovieLens dataset were imported into PostgreSQL using DBeaver.

After the import process was completed, several SQL queries were executed to verify that all records had been successfully loaded into the database.

These queries not only validated data integrity but also provided preliminary insights into the dataset.

6.1 Verification of Total Number of Movies

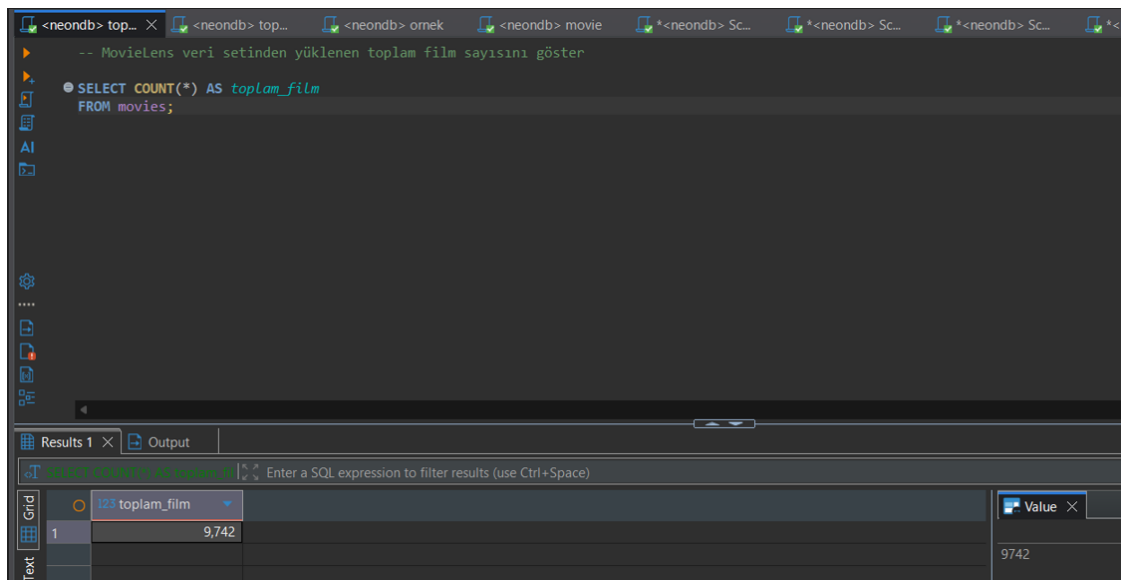
The following query was executed to determine the total number of movie records stored in the database.

```
SELECT COUNT(*) AS total_movies  
FROM movies;
```

Result:

9742 movies

This result confirms that all movie records from the MovieLens dataset were successfully imported into PostgreSQL.



6.2 Verification of Total Number of Ratings

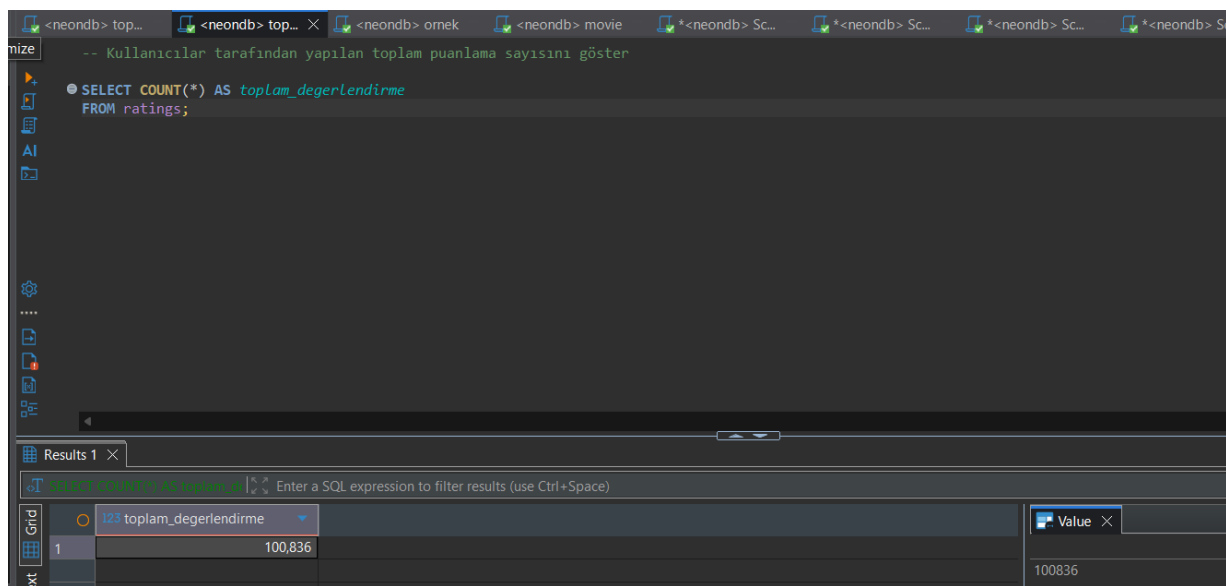
The following query was used to calculate the total number of ratings submitted by users.

```
SELECT COUNT(*) AS total_ratings
FROM ratings;
```

Result:

100,836 ratings

This result confirms that all records contained in ratings.csv were successfully imported into the database.



6.3 Examination of Sample Movie Records

To verify that the imported data was stored correctly, the following query was executed.

```
SELECT
    movieid,
    title,
    genres
FROM movies
LIMIT 10;
```

The query confirmed that movie titles and genre information were imported accurately.

-- Film tablosundan örnek kayıtları göster

```

SELECT
  movieid,
  title,
  genres
FROM movies
LIMIT 10;

```

movieid	title	genres
1	Toy Story (1995)	Adventure Animation Children's Comedy
2	Jumanji (1995)	Adventure Children Fantasy
3	Grumpier Old Men (1995)	Comedy Romance
4	Waiting to Exhale (1995)	Comedy Drama Romance
5	Father of the Bride Part II (1995)	Comedy
6	Heat (1995)	Action Crime Thriller
7	Sabrina (1995)	Comedy Romance
8	Tom and Huck (1995)	Adventure Children
9	Sudden Death (1995)	Action
10	GoldenEye (1995)	Action Adventure Thriller

6.4 Analysis of the Most Frequently Rated Movies

The following query was used to identify the movies that received the highest number of ratings from users.

```

SELECT
  m.title,
  COUNT(r.rating) AS rating_count
FROM movies m
JOIN ratings r
ON m.movieid = r.movieid
GROUP BY m.title
ORDER BY rating_count DESC
LIMIT 10;

```

The results indicate that the following movies received the largest number of ratings:

- Forrest Gump (1994)
- The Shawshank Redemption (1994)
- Pulp Fiction (1994)
- The Matrix (1999)

This analysis provides insight into which movies generated the highest level of user engagement.

-- Kullanıcılar tarafından en çok puanlanan filmleri listele

```

SELECT
  m.title,
  COUNT(r.rating) AS puan_sayisi
FROM movies m
JOIN ratings r
ON m.movieid = r.movieid
GROUP BY m.title
ORDER BY puan_sayisi DESC
LIMIT 10;

```

	A1 title	123 puan_sayisi
1	Forrest Gump (1994)	329
2	Shawshank Redemption, The (1994)	317
3	Pulp Fiction (1994)	307
4	Silence of the Lambs, The (1991)	279
5	Matrix, The (1999)	278
6	Star Wars: Episode IV - A New Hope (1977)	251
7	Jurassic Park (1993)	238
8	Braveheart (1995)	237
9	Terminator 2: Judgment Day (1991)	224
10	Schindler's List (1993)	220

6.5 Analysis of the Highest-Rated Movies

The following query lists movies that received at least 50 ratings and ranks them according to their average score.

```

SELECT
  m.title,
  ROUND(AVG(r.rating),2) AS average_rating,
  COUNT(*) AS vote_count
FROM movies m
JOIN ratings r
ON m.movieid = r.movieid
GROUP BY m.title
HAVING COUNT(*) > 50
ORDER BY average_rating DESC
LIMIT 10;

```

According to the results, the highest-rated movies include:

- The Shawshank Redemption (1994)
- The Godfather (1972)
- Fight Club (1999)
- Dr. Strangelove (1964)

This query identifies the most highly appreciated movies within the dataset.

```
-- En az 50 deęerlendirme almıř filmler arasında
-- ortalama puanı en yksek olanları listele
```

```
SELECT
    m.title,
    ROUND(AVG(r.rating),2) AS ortalama_puan,
    COUNT(*) AS oy_sayisi
FROM movies m
JOIN ratings r
ON m.movieid = r.movieid
GROUP BY m.title
HAVING COUNT(*) > 50
ORDER BY ortalama_puan DESC
LIMIT 10;
```

	A-Z title	123 ortalama_puan	123 oy_sayisi
1	Shawshank Redemption, The (1994)	4.43	317
2	Godfather, The (1972)	4.29	192
3	Fight Club (1999)	4.27	218
4	Cool Hand Luke (1967)	4.27	57
5	Dr. Strangelove or: How I Learned to Stop Worrying and Love the Bomb (1964)	4.27	97
6	Rear Window (1954)	4.26	84
7	Godfather: Part II, The (1974)	4.26	129
8	Goodfellas (1990)	4.25	126
9	Departed, The (2006)	4.25	107
10	Usual Suspects, The (1995)	4.24	204

6.6 Genre Distribution Analysis

The following query was executed to analyze the distribution of movie genres.

```
SELECT
    genres,
    COUNT(*) AS movie_count
FROM movies
GROUP BY genres
ORDER BY movie_count DESC
LIMIT 10;
```

This analysis helps to better understand the overall content structure of the dataset.

```
-- Trlere gre film sayılarını incele
```

```
SELECT
    genres,
    COUNT(*) AS film_sayisi
FROM movies
GROUP BY genres
ORDER BY film_sayisi DESC
LIMIT 10;
```

	A-Z genres	123 film_sayisi
1	Drama	1,053
2	Comedy	946
3	Comedy Drama	435
4	Comedy Romance	363
5	Drama Romance	349
6	Documentary	339
7	Comedy Drama Romance	276
8	Drama Thriller	168
9	Horror	167
10	Horror Thriller	135

6.7 Verification of the Embedding Column

To confirm that the pgvector extension was functioning correctly, the structure of the movies table was examined.

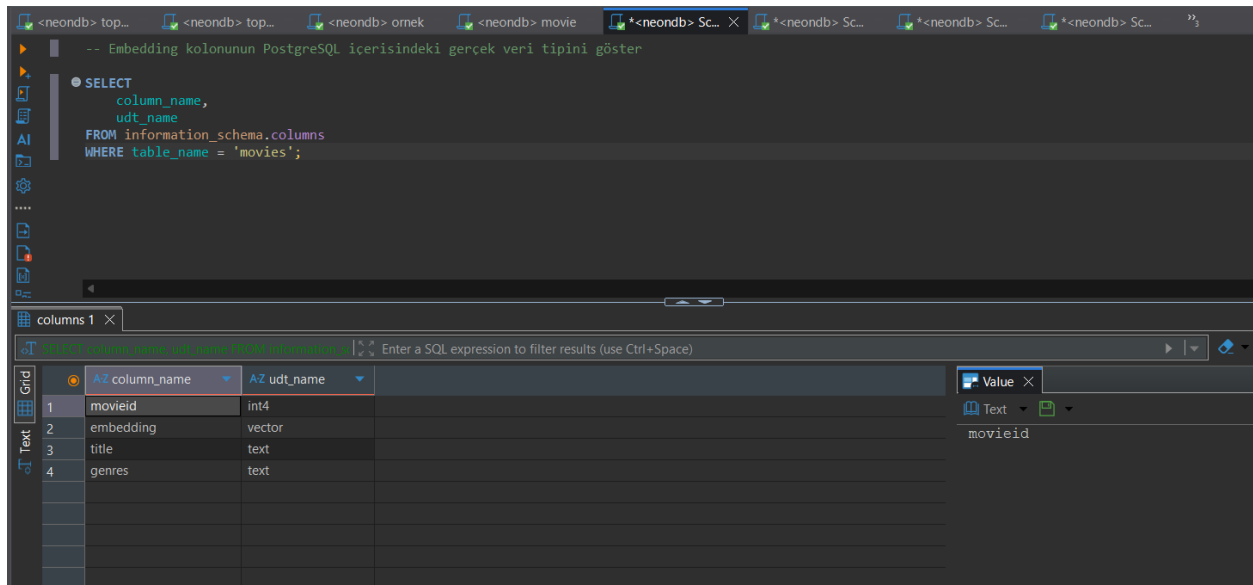
```
SELECT
    column_name,
    udt_name
FROM information_schema.columns
WHERE table_name = 'movies';
```

Result:

```
movieid    -> int4
embedding  -> vector
title      -> text
genres     -> text
```

This result demonstrates that PostgreSQL recognizes the embedding column as a genuine vector data type.

Therefore, the successful installation and operation of the pgvector extension have been verified.



The screenshot shows a PostgreSQL query editor with the following SQL query:

```
-- Embedding kolonunun PostgreSQL içerisindeki gerçek veri tipini göster
SELECT
    column_name,
    udt_name
FROM information_schema.columns
WHERE table_name = 'movies';
```

The results are displayed in a table with the following columns: column_name, udt_name, and Value. The results are as follows:

column_name	udt_name	Value
movieid	int4	
embedding	vector	
title	text	
genres	text	

7. pgvector Extension

The core technology used in this project is the pgvector extension.

Traditional PostgreSQL databases typically support data types such as:

- INTEGER
- VARCHAR
- DATE
- BOOLEAN

However, modern AI applications often require vector representations.

Example embedding:

[0.15, -0.22, 0.88, 0.34 ...]

Such data cannot be efficiently stored using standard PostgreSQL data types.

Therefore, the following command was used:

```
CREATE EXTENSION IF NOT EXISTS vector;
```

This command enables support for vector data types within PostgreSQL.

8. The Embedding Concept

An embedding is a mathematical representation of a text, object, or concept.

For example, the movie title:

Toy Story

can be transformed into a numerical vector representation such as:

[-0.0828, 0.0530, 0.0535, ...]

This vector serves as the semantic representation of the movie.

Similar movies tend to have similar vector representations.

As a result, recommendation systems and semantic search applications can be developed using these embeddings.

9. Creation of the Embedding Column

To store embedding information for movies, the following SQL command was executed:

```
ALTER TABLE movies
ADD COLUMN embedding VECTOR(384);
```

The value 384 represents the dimensionality of the embedding vectors generated by the selected AI model.

10. Embedding Generation Using Python

Python was used to generate embeddings from movie titles.

The selected model was:

all-MiniLM-L6-v2

This model is provided through the Sentence Transformers library.

Example:

```
model = SentenceTransformer("all-MiniLM-L6-v2")
```

The model converts movie titles into numerical vector representations suitable for similarity analysis and recommendation tasks.

11. Python Code

The following Python script was developed to generate embeddings from movie titles.

```
import pandas as pd
from sentence_transformers import SentenceTransformer

# Load the AI model
print("Loading model...")

model = SentenceTransformer("all-MiniLM-L6-v2")

# Read movie data
print("Reading CSV file...")

movies = pd.read_csv("movies.csv")

print(f"{len(movies)} movies found.")
```

```
# Select the first five movies as a sample
sample = movies.head(5)

# Convert movie titles into embedding vectors
embeddings = model.encode(
    sample["title"].tolist(),
    show_progress_bar=True
)

# Display the first few values of each embedding
for i, movie in enumerate(sample["title"]):
    print(movie)
    print(embeddings[i][:5])
    print()
```

12. Explanation of the Python Code

12.1 Importing the Pandas Library

```
import pandas as pd
```

This statement imports the Pandas library, which is used for reading CSV files and performing data processing operations.

12.2 Importing the Sentence Transformer Library

```
from sentence_transformers import SentenceTransformer
```

This library enables the generation of embeddings from textual data using pre-trained natural language processing models.

12.3 Loading the Model

```
model = SentenceTransformer("all-MiniLM-L6-v2")
```

This line downloads and loads the pre-trained all-MiniLM-L6-v2 model from Hugging Face.

The model is capable of converting movie titles into 384-dimensional vector representations.

12.4 Reading the CSV File

```
movies = pd.read_csv("movies.csv")
```

This statement loads the MovieLens movie dataset into memory.

12.5 Displaying the Number of Movies

```
print(f"{len(movies)} movies found.")
```

This line prints the total number of movies contained in the dataset.

Output:

```
9742 movies found.
```

12.6 Selecting Sample Movies

```
sample = movies.head(5)
```

The first five movies were selected for demonstration purposes.

This approach was used to verify that the system works correctly before applying the process to the entire dataset.

12.7 Generating Embeddings

```
embeddings = model.encode(  
    sample["title"].tolist(),  
    show_progress_bar=True  
)
```

This statement converts movie titles into vector representations using the artificial intelligence model.

For example:

Toy Story (1995)

may be transformed into an embedding such as:

```
[-0.0828, 0.0530, 0.0535, ...]
```

12.8 Displaying the Results

```
for i, movie in enumerate(sample["title"]):  
    print(movie)  
    print(embeddings[i][:5])
```

This loop prints the first five values of each generated embedding.

The output confirms that the AI model successfully generated embeddings for the selected movie titles.

13. Additional Explanation of the Python Workflow

The following operations were performed during the embedding generation process:

Loading the Pandas Library

Used for reading and processing CSV files.

Loading the Sentence Transformer Model

Used to convert movie titles into embeddings.

Reading the Movie Dataset

The MovieLens movie dataset was loaded into memory.

Selecting the First Five Movies

The first five records were selected as a demonstration sample.

```
sample = movies.head(5)
```

Generating Embeddings

The `model.encode()` function was used to transform movie titles into vector representations.

Displaying the Results

The first five values of each embedding vector were displayed for verification purposes.

14. Challenges Encountered

Several technical challenges were encountered throughout the project.

- Column mapping issues occurred during the import of CSV files into PostgreSQL.
- Neon database connection settings required configuration.
- The pgvector extension had to be verified and tested.
- The Python environment and required libraries needed to be installed and configured properly.

Resolving these challenges significantly contributed to the learning experience gained from this project.

15. Skills and Knowledge Gained

This project provided practical experience in the following areas:

- Cloud-based PostgreSQL deployment
- DBeaver database management
- CSV data import procedures
- Artificial intelligence and embedding concepts
- Sentence Transformers usage
- Recommendation system fundamentals
- pgvector extension implementation
- Data processing with Python

16. Real-World Applications

This approach can be applied to various real-world systems, including:

- Netflix recommendation systems
- Spotify music recommendation systems
- YouTube content recommendation systems
- E-commerce product recommendation platforms
- Semantic search applications
- AI-powered chatbot systems

17. Conclusion

In this project, the pgvector extension for PostgreSQL was explored and implemented.

The MovieLens dataset was imported into a PostgreSQL environment, movie information was stored in the database, and embeddings were generated using an artificial intelligence model.

The project demonstrates that PostgreSQL can be used not only for traditional relational database applications but also for modern artificial intelligence systems.

Through the pgvector extension, PostgreSQL becomes a powerful platform for supporting recommendation systems, semantic search applications, and other AI-driven solutions.

In conclusion, this project successfully demonstrates the integration of PostgreSQL and artificial intelligence technologies, providing a foundation for future recommendation and semantic search systems.

18. References

1. Harper, F. M., & Konstan, J. A. (2015). The MovieLens Datasets: History and Context. *ACM Transactions on Interactive Intelligent Systems (TIIS)*, 5(4), 1–19.
2. GroupLens Research. MovieLens Dataset. Retrieved from <https://grouplens.org/datasets/movielens/>
3. PostgreSQL Global Development Group. PostgreSQL Documentation. Retrieved from <https://www.postgresql.org/docs/>
4. pgvector Documentation. Retrieved from <https://github.com/pgvector/pgvector>
5. Reimers, N., & Gurevych, I. (2019). Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks. *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*.
6. Sentence Transformers Documentation. Retrieved from <https://www.sbert.net/>
7. Hugging Face. all-MiniLM-L6-v2 Model Documentation. Retrieved from <https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2>
8. Pandas Documentation. Retrieved from <https://pandas.pydata.org/docs/>
9. Neon Documentation. Retrieved from <https://neon.tech/docs>
10. DBeaver Documentation. Retrieved from <https://dbeaver.io/docs/>